DS 221 Introduction to Scalable Systems: Assignment 3

Parallelization of Histogram Calculation using OpenMP (7.5 points)

In this assignment, you will be calculating an histogram for a given array of numbers and for a given histogram interval. First generate an array of 2,00,000 (2 lakh) integer numbers between 1 and 50 as an input array. Have another array, called the histogram array, whose size will be equal 50/(histogram_interval). For example, if the histogram interval is 5, then the histogram array will be a 10-element array: the count of the input numbers between 1-5 will be stored in histogram_array[0], the count of the input numbers between 6-10 will be stored in histogram_array[1], and so on. Write a sequential program for creating the histogram array.

Parallelize the program using OpenMP where different threads process different elements of the input array. Both the input array and the output histogram arrays are shared among the threads. An OpenMP thread, when processing an element in the input array, increments the count of the corresponding element in the histogram_array. The main thread finally prints the counts of the histogram array.

Check the correctness of your parallel program by comparing the output with that of the sequential program. Show the timings of the sequential and the OpenMP version for different number of OpenMP threads (2, 4, 8, 16, 32) and for histogram intervals, namely, 2, 5, 10 and 25. For each experiment with a fixed number of threads, and for the sequential program, run 5 times, and obtain the average execution time across these 5 runs.

Prepare a report containing:

- Methodology
- Experimental setup
- Results containing execution times and speedup with the above options, both as tables and graphs with x and y axes properly labeled.
- Observations.

MPI: Merge Sort (7.5 points)

Implement a parallel merge sorting algorithm using a 1L integer array using MPI. Process 0 reads the elements from an input file and distributes to the other processes. Each process then performs a local sort. Then the processes get together in groups of two and merge their locally sorted arrays into an array sorted across all the elements of the two arrays. The representatives of these groups then do the same thing and so on, until the entire sorted array is formed in process 0.

Perform experiments for 8, 16, 32, 64 processes, find the times for the parallel sort in each case, and calculate the speedups with respect to the time for sequential quick sort.

Also, compare the times with another parallel version in which all the processes, after local sorting, send their local sorted arrays to process 0, which then does the merging.

Prepare a report with the methodology, execution times, and speedup graphs. For each experiment, run 5 times, and obtain the average execution time across these 5 runs.